

UNIVERSIDADE ESTADUAL DO CENTRO OESTE

Ciência da Computação

Programação Orientada a Objetos II - Trabalho II

Beatriz Oliveira De Jesus

11/11/2026

1. Diagnóstico do projeto original

O projeto original já possuía interface JavaFX com FXML, classes de modelo (Cliente, Pizza e Pedido) e DAOs responsáveis pela persistência no PostgreSQL. Entretanto, os Controllers instanciavam diretamente as classes DAO. Com isso, a interface acumulava responsabilidades de validação, coordenação das operações e acesso aos dados. No fluxo de pedidos, o Controller também consultava separadamente cliente, pizza e pedido e calculava o valor total. O cálculo estava fixo no Controller, dificultando a inclusão de novas regras de preço. O cadastro de pedidos também não contemplava adicionais.

2. Service Layer

Identificação e finalidade: A camada Service organiza operações e regras de negócio entre os Controllers e os DAOs. Foram criadas as classes ClienteService, PizzaService, PedidoService e ServiceException no pacote service.

Situação anterior: ClienteController, PizzaController e PedidoController chamavam diretamente métodos como buscarTodos(), inseri(), alteri() e exclui() dos DAOs. As validações de campos e valores também estavam concentradas nos Controllers.

Solução implementada: Os Controllers agora chamam os serviços. Os serviços validam os dados e encaminham as operações às interfaces DAO, mantendo o SQL e JDBC nas classes DAO.

Trecho significativo:

```
public void salvar(Pizza p) throws Exception {
    if (p == null || p.getNome() == null ||
            p.getNome().trim().isEmpty() ||
            p.getIngredientes() == null ||
            p.getIngredientes().trim().isEmpty())
        throw new ServiceException("Informe o nome e os ingredientes da pizza.");
    if (p.getPreco() <= 0) throw new ServiceException("O preço deve ser maior que zero.");
    if (p.getId() == 0) dao.inseri(p);
```

Justificativa e verificação: A validação deixa de depender da tela e passa a estar centralizada na camada de serviço. O CRUD de clientes e pizzas utiliza diretamente seus respectivos Services. O fluxo de pedidos utiliza os serviços por meio da Facade. Os Controllers não mantêm mais instâncias de DAO.

3. Facade

Identificação e finalidade: O padrão Facade fornece uma interface simplificada para um conjunto de operações. Foi implementado pela classe PedidoFacade, no pacote facade, que coordena ClienteService, PizzaService e PedidoService.

Situação anterior: O PedidoController precisava acessar três DAOs diferentes para carregar os dados dos combos, listar pedidos e salvar/excluir um pedido. A coordenação do fluxo ficava concentrada na interface.

Solução implementada: A Facade oferece métodos como listarClientes(), listarPizzas(), listarPedidos(), salvarPedido() e excluirPedido(). Ao salvar, consulta cliente e pizza, calcula o total e encaminha o objeto ao PedidoService.

Trecho significativo:

```
public void salvarPedido(int id, int clienteId,   1 usage
                         int pizzaId, int quantidade,
                         String status, String adicionais) throws Exception {
    Cliente cliente = clienteService.obter(clienteId);
    Pizza pizza = pizzaService.obter(pizzaId);
    double total = calcularTotal(pizza, quantidade, adicionais);
    Pedido pedido = new Pedido(id, cliente.getId(), pizza.getId(), quantidade, total, status);
    pedido.setAdicionais(adicionais == null ? "" : adicionais.trim());
    pedidoService.salvar(pedido);
}
```

Solução implementada: A Facade oferece métodos como listarClientes(), listarPizzas(), listarPedidos(), salvarPedido() e excluirPedido(). Ao salvar, consulta cliente e pizza, calcula o total e encaminha o objeto ao PedidoService.

Justificativa e verificação: O padrão Facade foi utilizado para simplificar a comunicação do PedidoController com os serviços, centralizando as operações relacionadas aos pedidos na classe PedidoFacade. Sua implementação foi verificada por meio das operações de listagem, cadastro e exclusão, incluindo o cálculo do valor total com adicionais e descontos. Com isso, o Controller ficou mais simples e o código mais organizado e fácil de manter.

4. Decorator

Identificação e finalidade: Decorator permite acrescentar responsabilidades a um objeto sem alterar sua classe original. Foram criados PrecoPizza (componente), PizzaBase (componente concreto) e AdicionalPizzaDecorator (decorador concreto).

Situação anterior: O preço era calculado apenas multiplicando o preço cadastrado da pizza pela quantidade. Não existia uma forma de incluir adicionais ao pedido sem alterar a lógica central do cálculo.

Solução implementada: A tela de pedidos recebeu o campo de adicionais, separados por vírgula. O Decorator envolve o preço base e

acrescenta R$ 3,50 por adicional. A informação é salva na coluna pedido.adicionais

Trecho significativo:

```
public double calcularPreco() { return pizza.calcularPreco()  2 usages
        + quantidadeAdicionais * PRECO_ADICIONAL; }

private int contar(String texto) {  1 usage
    if (texto == null || texto.trim().isEmpty()) return 0;
    return (int) java.util.Arrays.stream(texto.split( regex: ","))
            .map(String::trim).filter( String s -> !s.isEmpty()).count();
}
```

Solução implementada: A tela de pedidos recebeu o campo de adicionais, separados por vírgula. O Decorator envolve o preço base e acrescenta R$ 3,50 por adicional. A informação é salva na coluna pedido.adicionais.

Justificativa e verificação: O padrão Decorator permite adicionar funcionalidades a um objeto sem modificar sua classe original. No projeto, ele foi utilizado para incluir adicionais ao preço da pizza, acrescentando R$ 3,50 por adicional informado. A implementação foi verificada pelo cálculo do valor do pedido e pelo salvamento dos adicionais na coluna pedido.adicionais do banco de dados.

5. Strategy

Identificação e finalidade: Strategy permite encapsular algoritmos intercambiáveis. A interface PoliticaPreco define o cálculo; PrecoNormal e DescontoQuantidade são estratégias concretas.

Situação anterior: O cálculo estava fixo em uma multiplicação no Controller, sem separar políticas comerciais.

Solução implementada: A Facade seleciona PrecoNormal para pedidos com menos de cinco unidades e DescontoQuantidade para pedidos com

cinco ou mais unidades. A estratégia promocional aplica 10% de desconto ao subtotal, já considerando os adicionais.

Trecho significativo:

```
public class DescontoQuantidade implements PoliticaPreco {  1 usage
    public double calcular(double valorUnitario, int quantidade) {  1 usage
        double total = valorUnitario * quantidade;
        return quantidade >= 5 ? total * 0.90 : total;
    }
    public String descricao() { return "Desconto de 10% para 5 ou mais pizzas"; }  no usages
}
```

Solução implementada: A Facade seleciona PrecoNormal para pedidos com menos de cinco unidades e DescontoQuantidade para pedidos com cinco ou mais unidades. A estratégia promocional aplica 10% de desconto ao subtotal, já considerando os adicionais.

Justificativa e verificação: O padrão Strategy foi utilizado para separar as diferentes políticas de cálculo do preço, evitando que as regras promocionais ficassem fixas no Controller. A implementação foi verificada por meio de pedidos com diferentes quantidades, confirmando que pedidos com menos de cinco unidades utilizam o preço normal e pedidos com cinco ou mais unidades recebem 10% de desconto sobre o subtotal, incluindo os adicionais.

GitHub: https://github.com/Bia8791/PizzariaPOOII.git